

Lecture 01 | July, 2026

Evaluation

Instructor: Sithvothy Kiv

Slides Credit: Josh Hug

Evaluation

- **Intro**
 - The evaluation setup
 - Generating Ground Truth Data
 - Loading the documents
 - Generating questions with structured output
 - Reusable utilities
 - Tracking cost
 - Generating Ground Truth for All Documents
 - Parallel processing

Intro

In the previous modules, we built search engines and RAG pipelines. We tried different approaches: keyword search with [minsearch](#), [vector search](#), [agents with function calling](#). But we never answered the obvious question of which one is actually better.

We could try a few queries by hand and see what looks good. That's fine for a quick sanity check, but it doesn't scale, and it doesn't give us a number to compare. We need a systematic way to tell whether one approach beats another.

Intro

That's what evaluation is for. And it's worth saying up front: of everything in this course, evaluation is the part that matters most. It's also the most tedious. But it's the only way to be sure your system works. And it's how you keep it working as you change prompts and swap models.

The evaluation setup

- Intro
- **The evaluation setup**
- Generating Ground Truth Data
- Loading the documents
- Generating questions with structured output
- Reusable utilities
- Tracking cost
- Generating Ground Truth for All Documents
- Parallel processing

The evaluation setup

For search evaluation, we need a dataset of questions where we know which document is the correct answer.

We'll use an LLM to generate these questions from our FAQ data.

The approach works like this:

- A = the original answer in the FAQ
- Q^* = a question generated from that answer by an LLM
- We send Q^* through our search and check if the original document appears in the results

The evaluation setup

For RAG evaluation, we go one step further:

- A = the original answer in the FAQ
- Q^* = a question generated from that answer by an LLM
- A' = the answer produced by our RAG system when given Q^*
- We compare A' with A to see if the system produced the right answer

This is the $A \rightarrow Q^* \rightarrow A'$ pattern.

We know the answer for each generated question because we created the question from that answer.

The evaluation setup

With evaluation, we can:

- Compare different search methods (minsearch vs vector search vs hybrid)
- Tune parameters (boost values, number of results, prompt templates)
- Compare different LLMs (gpt-5.4-mini vs others)
- Track improvements over time

There are two types of evaluation:

- Offline evaluation: run the system on a test dataset and compute metrics
- Online evaluation: collect feedback from real users in production

Offline evaluation is what we do before putting changes in front of users. It lets us compare search settings, prompts, or models on the same dataset. Online evaluation happens after deployment. It uses real traffic, feedback, logs, and dashboards to monitor quality.

The evaluation setup

In this module, we focus on offline evaluation. We'll generate a test dataset, run our search and RAG systems on it, and measure how well they perform.

Synthetic data (information created artificially by computers and algorithms) is a good starting point when you don't have real user data. But generated questions can be too similar to the original FAQ text, which inflates the metrics.

As soon as you can, start collecting real user queries and use them to validate your evaluation framework.

The evaluation setup

We'll cover three levels of evaluation:

1. Search evaluation: does the search return the right documents?
2. RAG evaluation: does the LLM generate good answers?
3. Agent evaluation: does the agent use tools efficiently?

Most of our time goes to search, and that's on purpose.

Everything else depends on it: if retrieval brings back the wrong documents, no prompt or model can rescue the answer.

So we test search on its own first, then evaluate the full pipeline on top of it.

The evaluation setup



Search

Evaluate whether the search returns the correct, relevant documents.

Metrics: Hit Rate, MRR (Mean Reciprocal Rank)



RAG Quality

Evaluate if the LLM generates high-quality, accurate answers based on context.

Method: LLM-as-a-judge



Agents

Evaluate the tool-call execution path and the final answer resolution.

Focus: Final answer & tool-call trajectory

Let's start with generating the test data we need.

Generating Ground Truth Data

- Intro
- The evaluation setup
- **Generating Ground Truth Data**
- Loading the documents
- Generating questions with structured output
- Reusable utilities
- Tracking cost
- Generating Ground Truth for All Documents
- Parallel processing

Generating Ground Truth Data

To evaluate search, we need a dataset of queries where we know which document is the correct answer. This is called ground truth (or gold standard) data.

For each query in our ground truth dataset, we know which document in the knowledge base is relevant. When we run a search, we check whether the results include the correct document.

Generating Ground Truth Data

There are several ways to get ground truth data:

- Human annotators look at documents and write queries (best quality, expensive)
- Collect real user queries and label them (requires a running system)
- Generate synthetic data with an LLM (what we'll do)

We don't have a production system yet, so we'll use an LLM to generate questions.

For each FAQ document, we ask the LLM to create 5 questions that this document would answer. Then we know that for each generated question, the source document is the correct answer.

Loading the documents

- Intro
- The evaluation setup
- Generating Ground Truth Data
- **Loading the documents**
- Generating questions with structured output
- Reusable utilities
- Tracking cost
- Generating Ground Truth for All Documents
- Parallel processing

Loading the documents

We'll use helper files from module 01 and this module.

```
from ingest import load_faq_data

documents = load_faq_data()

documents_llm = []

for doc in documents:
    if doc["course"] == "llm":
        documents_llm.append(doc)

len(documents_llm)
```


We'll use helper files from module 01 and this module.

```
doc = documents[0]
print(doc["id"])
print(doc["question"])
print(doc["answer"])
```

The ID becomes the label in our ground truth dataset. We generate questions from a document, so we know that this document holds the answer. Later, search evaluation checks whether search brings back the document with this ID.

This is why every record needs a stable ID. If you can't uniquely identify a document, you can't tell whether search retrieved the right one. When you build your own evaluation set, assign an ID to each record in your knowledge base first.

Generating question with structured output

- Intro
- The evaluation setup
- Generating Ground Truth Data
- Loading the documents
- **Generating questions with structured output**
- Reusable utilities
- Tracking cost
- Generating Ground Truth for All Documents
- Parallel processing

We use an LLM to generate questions for each document.

This is the first time we're using structured output in the course. With structured output, we ask the LLM to return data in a specific format instead of free-form text. For example, instead of getting a paragraph that contains questions, we can ask for a Python object with a questions field.

This is useful when code will process the output. The model returns the same structure every time. We can access the generated questions directly instead of parsing text manually.

Generating questions with structured output

We want the output as a list of strings, so we define that structure with a Pydantic model:

```
from pydantic import BaseModel

class Questions(BaseModel):
    questions: list[str]
```

The instructions for the LLM:

```
data_gen_instructions = """
You emulate a student who's taking our course.
Formulate 5 questions this student might ask based on a FAQ record. The
record
should contain the answer to the questions, and the questions should be
complete and not too short.
If possible, use as fewer words as possible from the record.

The output should resemble how people ask questions
on the internet. Not too formal, not too short, not too long.
""".strip()
```

Generating questions with structured output

We ask the LLM to use different wording from the original document. This makes the evaluation more realistic - real users won't phrase their questions the same way as the FAQ.

Call the LLM for one document:

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
openai_client = OpenAI()
```

Prepare the document as JSON and Create the messages:

```
import json

user_prompt = json.dumps(doc)
messages = [
    {"role": "developer", "content": data_gen_instructions},
    {"role": "user", "content": user_prompt}
]
```


Generating questions with structured output

Until now we called `responses.create` and read `response.output_text`. For structured output we switch to `responses.parse` and pass `text_format=Questions`, which tells the API to return our class instead of free text.

Call the model:

```
response = openai_client.responses.parse(  
    model="gpt-5.4-mini",  
    input=messages,  
    text_format=Questions  
)
```

Generating questions with structured output

The parsed object is available in `response.output_parsed`:

```
result = response.output_parsed  
print(result.questions)
```

Reusable utilities

- Intro
- The evaluation setup
- Generating Ground Truth Data
- Loading the documents
- Generating questions with structured output
- **Reusable utilities**
- Tracking cost
- Generating Ground Truth for All Documents
- Parallel processing

We'll need this pattern again in other evaluation sections today, so we put it in a reusable helper.

It contains helper functions we'll reuse in this module:

- `llm_structured`: calls the OpenAI API with structured output
- `llm_structured_retry`: retries structured-output calls when a request fails
- `calc_price`: calculates the price from token usage
- `calc_total_price`: calculates the total price from multiple usage objects
- `map_progress`: runs work in parallel and tracks progress. Next, We'll use it.

We'll need this pattern again in other evaluation sections today, so we put it in a reusable helper.

It contains helper functions we'll reuse in this module:

```
from evaluation_utils import llm_structured
result, usage = llm_structured(
    openai_client,
    data_gen_instructions,
    user_prompt,
    Questions
)

print(result.questions)
```

Tracking cost

- Intro
- The evaluation setup
- Generating Ground Truth Data
- Loading the documents
- Generating questions with structured output
- Reusable utilities
- **Tracking cost**
- Generating Ground Truth for All Documents
- Parallel processing

The response also contains token usage:

```
usage.input_tokens, usage.output_tokens

from evaluation_utils import calc_price
cost = calc_price(usage)

cost
```

Now convert these questions into ground truth records:

```
records = []

for q in result.questions:
    records.append({
        "question": q,
        "document": doc["id"]
    })

records
```


Each record has two fields:

- question: the question generated by the LLM
- document: the ID of the FAQ document that should answer the question

The document field connects the generated question to the document that contains the answer. Later, when we evaluate search, we'll ask the search engine the generated question. Then we'll check if it retrieves the document with this ID.

We now know how to generate and store questions for one document.

Next, we'll run this for all LLM course FAQ documents and save the full ground truth dataset.

Generation Ground Truth

- Intro
- The evaluation setup
- Generating Ground Truth Data
- Loading the documents
- Generating questions with structured output
- Reusable utilities
- Tracking cost
- **Generating Ground Truth for All Documents**
- Parallel processing

We already generated questions for one document and converted them into ground truth records.

We want to do the same thing for every document in the FAQ dataset. For each document, we generate questions and save them as ground truth records.

For this part, we'll use tqdm for progress bars and pandas for saving the final CSV.

Generating Ground Truth for All Documents

If you don't have them installed yet, add them first:

```
uv add tqdm pandas
```

The processing function takes one document and turns it into ground truth records.

For each document, we:

- convert the document to JSON so we can send it to the LLM
- ask the LLM to return a Questions object
- create one ground truth record for each generated question

Each record contains the generated question and the ID of the document that should answer the question.

When we send many requests, one of them might fail. We don't want the entire batch to fail because of one temporary error.

Import the retry helper from `evaluation_utils.py`:

```
from evaluation_utils import llm_structured_retry
```

`llm_structured` makes one structured-output call. `llm_structured_retry` wraps the same call in a retry loop. If one request fails because of a temporary API or network issue, it waits briefly and tries again.

Use it in the processing function:

```
def generate_ground_truth(doc):  
    user_prompt = json.dumps(doc)  
  
    out, usage = llm_structured_retry(  
        openai_client,  
        data_gen_instructions,  
        user_prompt,  
        Questions  
    )  
  
    results = []  
  
    for q in out.questions:  
        results.append({  
            "question": q,  
            "document": doc["id"]  
        })  
  
    return results, usage
```

Generating Ground Truth for All Documents

Import tqdm and run the loop:

```
from tqdm.auto import tqdm

ground_truth = []
usages = []

for doc in tqdm(documents[:5]):
    records, usage = generate_ground_truth(doc)
    ground_truth.extend(records)
    usages.append(usage)
```

This works, but it runs one LLM call after another. Running it for all documents this way would take too long.

Parallel processing

- Intro
- The evaluation setup
- Generating Ground Truth Data
- Loading the documents
- Generating questions with structured output
- Reusable utilities
- Tracking cost
- Generating Ground Truth for All Documents
- **Parallel processing**

Running the calls one after another wastes most of the time waiting on the network. Each request just sits there until OpenAI responds, so we can fire several at once and wait on them together. We process the documents in parallel and track progress while the requests run.

One caution: don't open too many connections at once, or you'll hit the provider's rate limits. Five or six workers is a safe default here.

Import ThreadPoolExecutor:

```
from concurrent.futures import ThreadPoolExecutor  
from evaluation_utils import map_progress
```

This submits one job per document, updates the progress bar when a job finishes, and collects the results. If you want a more detailed explanation of ThreadPoolExecutor and futures, ask ChatGPT to walk through this helper line by line.

Then replace the loop with the parallel version:

```
with ThreadPoolExecutor(max_workers=6) as pool: results =  
map_progress(pool, documents, generate_ground_truth)
```

`generate_ground_truth` returns two things for each document: the generated records and the token usage.

Split those into separate lists:

```
ground_truth = []
usages = []
for records, usage in results:
    ground_truth.extend(records)
    usages.append(usage)
len(ground_truth)
```

Calculate the total cost:

```
from evaluation_utils import calc_price
total_cost = 0.0
for usage in usages:
    cost = calc_price(usage)
    total_cost = total_cost + cost["total_cost"]
total_cost
```

We'll calculate total cost several times in this module, so the utility file has a helper for it:

```
from evaluation_utils import calc_total_price  
calc_total_price(usages)
```

Create a dataframe so we can look at the records as a table and save them as a CSV file.

Create the dataframe:

```
import pandas as pd
df_ground_truth = pd.DataFrame(ground_truth)
```


Because we generated the questions from specific documents, we know which document is correct for each question. We now have the ground truth we need for evaluation.

Save it for later use:

```
df_ground_truth.to_csv("data/ground_truth-new.csv", index=False)
```

Now we have questions with known correct documents. In the next lesson, we'll run search for these questions and check whether the correct documents appear in the results.